



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

AI-POWERED FAKE NEWS AND MISINFORMATION DETECTION SYSTEM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the course of

ARTIFICIAL INTELLIGENCE

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Asha Sarthiwika (192311066)

Under the Supervision of

Dr. Krishna Kumar Kathiresan,

Professor

**SIMATS ENGINEERING,
SAVEETHA SCHOOL OF ENGINEERING**

SIMATS, Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Saveetha School of Engineering

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “AI-Powered Fake News and Misinformation Detection System” has been carried out by V. Asha Sarthiwika (192311066), under the guidance of Dr. Krishna Kumar Kathiresan, Professor, and is submitted in partial fulfilment of the requirements for the Bachelor of Engineering in Computer Science and Engineering at Saveetha School of Engineering, SIMATS. The work presented in this report represents a team-based engineering solution involving natural language processing, machine learning, software implementation, testing, analysis, and web deployment.

COURSE COORDINATOR

Dr. Krishna Kumar Kathiresan

Professor

Generate AI

Saveetha School of Engineering

SIMATS

COURSE FACULTY

Dr. Krishna Kumar Kathiresan

Professor

Generate AI

Saveetha School of Engineering

SIMATS

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, V. Asha Sarthiwika (192311066) of the Department of Computer Science and Engineering, Saveetha School of Engineering, SIMATS, hereby declare that the Capstone Project Work entitled “AI-Powered Fake News and Misinformation Detection System” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with engineering ethics and academic integrity.

All external literature, datasets, software libraries, development tools, and AI-assisted resources used during the project are acknowledged in the References section. The team confirms that experimental results will be reported only from actual execution and will not be fabricated, falsified, or manipulated.

The system is designed as an AI-assisted screening and decision-support tool. Its classification output is not claimed to be absolute proof of truth or falsity, and important claims should be independently verified using trusted sources.

Place: Chennai

Date: September 2026

Asha Sarthiwika.V(192311066)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
V. Asha Sarthiwika / 192311066	Dataset preparation and NLP preprocessing	Dataset inspection, cleaning, text normalization and preprocessing pipeline	Preprocessing test cases, input validation and data-quality checks	Ch. 1, Ch. 3 and preprocessing sections	33%

The contribution percentages are a draft team allocation and should be confirmed by all members before final submission. Each member is expected to actively participate in development, testing, documentation, presentation, and viva activities.

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh, for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr. Sri Ramya P, for continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr. Krishna Kumar Kathiresan, Professor, for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members, and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

A. Sarthiwika (192311066)

ABSTRACT

The rapid growth of social media, online news platforms, messaging applications, and digital communication has increased the speed at which misleading information can spread. Manual verification of every article is time-consuming and difficult to scale. The AI-Powered Fake News and Misinformation Detection System is proposed as an automated screening tool that analyzes textual news content and predicts whether the content is likely to be REAL NEWS or FAKE NEWS.

The proposed system uses Natural Language Processing (NLP) to clean and normalize news text, followed by Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction. Unlike the Naive Bayes, Logistic Regression, and SVM algorithms used in the earlier project version, this version intentionally selects the Passive-Aggressive Classifier as the primary and unique classification algorithm. Passive-Aggressive learning is suitable for high-dimensional text classification and can update its decision boundary efficiently when training examples are processed. This makes it an appropriate algorithm for a lightweight misinformation-screening prototype.

The system follows a modular pipeline: dataset collection and validation, text preprocessing, TF-IDF vectorization, Passive-Aggressive model training, evaluation, Flask-based web deployment, and real-time news classification. The interface is designed to display the processing stages and prediction result clearly. Accuracy, precision, recall, F1-score, and confusion-matrix analysis are used for validation.

The supplied source materials establish the project dataset and previous interface structure, but they do not contain a verified execution result for the newly selected Passive-Aggressive algorithm. Therefore, this report does not invent a performance value. The final numerical metrics must be obtained by running the new training implementation on the actual project dataset. The expected contribution is a lightweight, explainable, and deployable AI-assisted screening system that helps users identify potentially misleading news while emphasizing that automated classification is not a substitute for independent fact-checking.

Keywords: Fake News Detection, Misinformation, Natural Language Processing, TF-IDF, Passive-Aggressive Classifier, Flask

TABLE OF CONTENTS

Sl. No.	Title	Page No.
1	Certificate from the Supervisor	ii
2	Declaration by the Candidate	iii
3	Individual Contribution Statement	iv
4	Acknowledgement	v
5	Abstract	vi
6	List of Figures	vii
7	List of Tables	viii
8	List of Abbreviations / Symbols	ix
9	Chapter 1: Introduction and Engineering Problem	1
10	Chapter 2: Literature Review and Existing Solutions	6
11	Chapter 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	11
12	Chapter 4: Implementation, Testing & Results, Project Management	18
13	Chapter 5: Conclusion, Future Work and Reflection	26
14	References	31
15	Appendices	33

Note: The page numbers above are a structured guide based on the planned report layout. The page numbers in the footer are generated automatically by Microsoft Word and may shift slightly if content or formatting is changed.

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1.1	Complete AI-Powered Fake News Detection Workflow	4
Figure 3.1	Dataset Overview and Class Distribution Dashboard	14
Figure 3.2	Text Preprocessing and TF-IDF Interface	15
Figure 3.3	Model Training and Performance Dashboard	16
Figure 4.1	Prediction Interface and Classification Result	21
Figure 4.2	Development Environment / Project Structure	23

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1.1	Project Objectives and Measures	3
Table 2.1	Comparison of Existing and Proposed Approaches	8
Table 3.1	Stakeholder Requirements	12
Table 3.2	Functional and Non-Functional Requirements	13
Table 3.3	Design Specifications	14
Table 3.4	Alternative Solution Evaluation	17
Table 3.5	Risk Register	19
Table 4.1	Software Tools and Resources	20
Table 4.2	Test Plan and Verification	22
Table 4.3	Performance Results Template	24
Table 4.4	Objective Achievement	25
Table 4.5	Team Roles and Contributions	27

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	-
NLP	Natural Language Processing	-
TF-IDF	Term Frequency–Inverse Document Frequency	-
ML	Machine Learning	-
PAC	Passive-Aggressive Classifier	-
CSV	Comma-Separated Values	-
API	Application Programming Interface	-
UI	User Interface	-
HTTP	Hypertext Transfer Protocol	-
TP / TN	True Positive / True Negative	counts
FP / FN	False Positive / False Negative	counts

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Digital news is now consumed through websites, social-media feeds, messaging applications, and online communities. This rapid information flow creates an environment in which inaccurate or deliberately misleading claims can be distributed faster than they can be manually verified. Fake news and misinformation can influence public opinion, create confusion, damage reputations, and affect decisions.

Artificial Intelligence provides a practical way to perform an initial automated screening of large volumes of text. In a text-classification system, news articles are transformed into numerical features and a machine-learning classifier learns patterns associated with labelled REAL and FAKE examples. The proposed project focuses on textual classification rather than claiming to establish factual truth.

The project adopts an NLP pipeline consisting of data validation, text cleaning, token normalization, TF-IDF feature extraction, classification, evaluation, and web deployment. The selected classifier is the Passive-Aggressive Classifier, chosen as a different algorithm from Naive Bayes, Logistic Regression, and SVM used in the earlier project version.

1.2 Need for the Project

- Large quantities of online news make complete manual screening impractical.
- Users need a fast first-level warning before accepting or sharing suspicious content.
- Raw text contains noise such as URLs, punctuation, HTML fragments, numbers, and inconsistent casing.
- Machine-learning classification can learn statistical text patterns from labelled examples.
- A web interface makes the trained model accessible without requiring the user to run Python code directly.
- The system can support awareness and prioritization of articles for further verification.

1.3 Problem Statement

Design and implement an AI-powered software system that accepts labelled news data and user-provided news text, preprocesses the text, converts it into TF-IDF features, and uses a Passive-Aggressive Classifier to predict whether the article is likely to be real or fake. The system must provide clear results, handle invalid input safely, support model training and testing, and expose the prediction workflow through a Flask web application.

The engineering challenge is to balance classification effectiveness, computational cost, speed, usability, interpretability, data quality, and responsible use of predictions. Because misinformation can be context-dependent, the system must not present a statistical prediction as absolute proof of truth or falsity.

1.4 Project Aim

To develop a lightweight AI-assisted fake news and misinformation screening system that uses NLP and a Passive-Aggressive text-classification algorithm to identify potentially misleading news articles and present the prediction through an accessible web interface.

1.5 Project Objectives

1. To collect, inspect, and validate a labelled dataset containing REAL and FAKE news records.
2. To preprocess raw news text by normalizing case and removing unnecessary noise.
3. To convert cleaned text into numerical TF-IDF feature vectors.
4. To implement the Passive-Aggressive Classifier as the primary classification algorithm.
5. To evaluate the classifier using accuracy, precision, recall, F1-score, and confusion matrix.
6. To integrate the trained model into a Flask web application.
7. To provide clear REAL/FAKE prediction output and suitable input validation.
8. To document limitations and responsible-use considerations for misinformation detection.

Objective	Measure / Evidence
Dataset preparation	Validated labelled CSV and data-quality summary
NLP preprocessing	Cleaned text and preprocessing output
Feature extraction	TF-IDF sparse feature matrix
AI model	Trained Passive-Aggressive Classifier
Evaluation	Accuracy, precision, recall, F1-score and confusion matrix
Deployment	Working Flask interface and prediction endpoint

1.6 Scope

Included: labelled text-news dataset handling, preprocessing, TF-IDF feature extraction, Passive-Aggressive classification, evaluation, Flask deployment, user input validation, and result visualization.

Excluded: automatic fact verification against every external source, real-time social-media scraping, multilingual translation, image/video deepfake analysis, audio analysis, and replacement of professional fact-checking. The supplied source project also identifies multimedia verification and cross-lingual processing as outside the initial scope. □filecite□turn10file2□L157-L164□

Assumptions: the training dataset contains sufficiently reliable REAL/FAKE labels; the input is primarily English text; the training and deployment environments have compatible Python and library versions; and the trained model is used only within the distribution represented by the training data.

1.7 Expected Engineering Outcome

The expected outcome is a modular software prototype that accepts news text, applies the same preprocessing and feature transformation used during training, produces a Passive-Aggressive classification, and displays the result through a Flask web application. The system should also provide evidence of testing and allow the team to reproduce the model-training process.

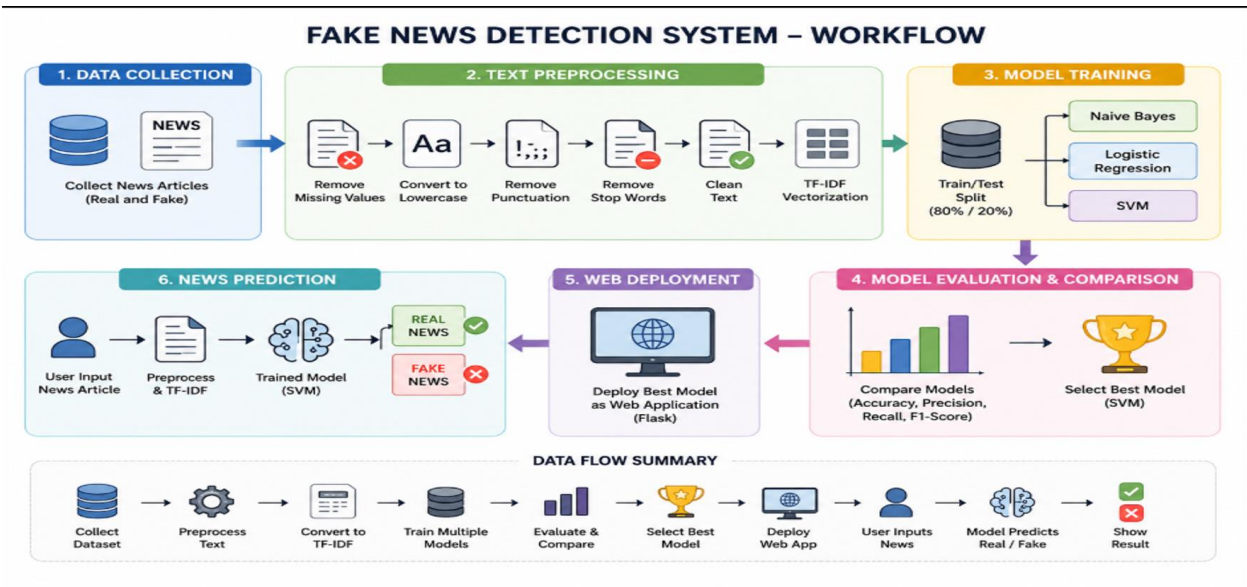


Figure 1.1: Complete AI-Powered Fake News and Misinformation Detection Workflow

The workflow is adapted to the new project title and selected algorithm. The original supplied workflow illustrates data collection, preprocessing, TF-IDF, train/test splitting, model training, evaluation, web deployment, and prediction. For this version, Passive-Aggressive Classification is used as the selected model instead of the earlier Naive Bayes / Logistic Regression / SVM set.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The source material reviewed for this report discusses fake-news detection, NLP, TF-IDF, classical machine learning, deep learning, and web deployment. The earlier project report identifies structured searches across IEEE Xplore, ScienceDirect, ACM Digital Library, and Google Scholar using terms such as fake news detection, natural language processing, TF-IDF machine learning, and social-media misinformation. □filecite□turn11file1□L44-L48□

For the revised project, the literature discussion is retained at the conceptual level while the implementation is changed to a Passive-Aggressive Classifier. This keeps the report consistent with the requirement that the new project should use an algorithm not used in the earlier implementation.

2.2 Review of Existing Work

Bhardwaj and Rani (2021) surveyed machine-learning and deep-learning approaches for fake news detection and discussed the efficiency of traditional classifiers with text feature engineering. Khan et al. (2021) examined machine-learning models for online fake-news detection and highlighted the importance of feature engineering and preprocessing. Lazer et al. (2018) described fake news as a socio-technical problem requiring interventions across technology and information ecosystems. These sources are included in the supplied project material. □filecite□turn11file1□L46-L48□

Transformer-based approaches such as BERT can capture richer contextual information, but they typically require greater computational and deployment resources. The earlier source project therefore considered classical text classification suitable for a lightweight web prototype. □filecite□turn10file5□L331-L337□

The present project introduces Passive-Aggressive learning as a deliberate alternative. The algorithm is designed for online or large-scale learning and updates the model strongly when a training example is misclassified while remaining passive when the current model already satisfies the desired classification constraint. Its ability to work with high-dimensional sparse features makes it a reasonable candidate for TF-IDF-based news text.

2.3 Comparative Analysis

Existing / Proposed Approach	Advantages	Limitations	Relevance
Manual fact-checking	High-quality contextual verification by experts	Slow and difficult to scale	Provides human verification after automated screening
Naive Bayes	Fast and simple for text classification	Earlier project algorithm; simplifying independence assumptions	Not selected in the revised project
Logistic Regression	Strong linear text baseline and interpretable	Earlier project algorithm	Not selected in the revised project
SVM	Strong performance on sparse high-dimensional text	Earlier project algorithm; margin model can be less suitable for online updating	Not selected in the revised project
Transformer / BERT	Rich semantic and contextual representation	Higher computational and deployment requirements	Advanced future benchmark

Passive-Aggressive Classifier	Fast, lightweight, suitable for sparse text and incremental learning	Sensitive to noisy labels and hyperparameter choice	Selected unique algorithm for this project
-------------------------------	--	---	--

2.4 Research / Engineering Gap

The earlier implementation relied on three familiar supervised classifiers. The revised project addresses the algorithmic gap by selecting a different classifier that is particularly relevant to streaming or continuously updated text data. A practical misinformation system may need to adapt to changing vocabulary, emerging topics, and new patterns. Passive-Aggressive learning provides a pathway toward such incremental adaptation without requiring a complete redesign of the application architecture.

A second gap is the difference between classification and factual verification. A text classifier can learn correlations in wording and source-related patterns but cannot independently prove every claim. The proposed system therefore positions its output as a screening signal that should be followed by human or trusted-source verification.

2.5 Proposed Contribution

- Introduces Passive-Aggressive classification as the primary algorithm, distinct from Naive Bayes, Logistic Regression, and SVM.
- Combines TF-IDF sparse text representation with a lightweight classifier suitable for rapid inference.
- Provides an end-to-end pipeline from dataset preparation to browser-based prediction.
- Makes preprocessing visible to support transparency and debugging.
- Includes responsible-use guidance so users do not treat the prediction as absolute proof.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student / Developer	Easy dataset loading, reproducible training, readable logs and model files
End User	Simple interface for entering news text and receiving a clear prediction
Faculty / Evaluator	Evidence of algorithm, testing, metrics, screenshots and engineering decisions
System Administrator	Safe handling of uploaded files and controlled model storage

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Load labelled news dataset	Functional	CSV is read and REAL/FAKE labels are detected
Preprocess news text	Functional	Cleaning pipeline produces normalized text
Generate TF-IDF features	Functional	Sparse feature matrix is created
Train Passive-Aggressive model	Functional	Model completes training without runtime error
Predict REAL/FAKE	Functional	Valid non-empty input returns one class
Show evaluation metrics	Functional	Accuracy, precision, recall and F1-score are recorded
Input validation	Non-Functional	Blank or malformed input is rejected safely
Response speed	Non-Functional	Prediction should complete quickly on local deployment
Maintainability	Non-Functional	Dataset, model, vectorizer and Flask code remain modular
Security	Non-Functional	Uploaded content is validated and unnecessary personal data is not stored

3.2 Constraints & Applicable Standards

The project is a software and machine-learning prototype. The most relevant constraints are dataset quality, limited student computing resources, model generalization, usability, secure input handling, reproducibility, and responsible interpretation. The first-format document specifically asks for applicable standards rather than generic statements. The source project identifies ISO/IEC 25010, ISO/IEC 27001, IEEE 29148, and IEEE 29119 as relevant engineering references.

□ filecite□turn10file6□L355-L375□

Standard / Code	Requirement	How Applied
ISO/IEC 25010	Software product quality characteristics	Consider functionality, performance, usability, reliability, security, maintainability and portability
ISO/IEC 27001	Information security management principles	Use responsible handling of uploaded text, datasets and stored model artifacts
ISO/IEC/IEEE 29148	Requirements engineering	Define functional/non-functional requirements and system boundaries

ISO/IEC/IEEE 29119	Software testing	Structure unit, integration, system and validation tests
--------------------	------------------	--

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Dataset	Labelled REAL and FAKE news records; use actual project CSV	records	High
Text representation	TF-IDF	features	High
Classifier	Passive-Aggressive Classifier	algorithm	High
Split	80 / 20 recommended for reproducible evaluation	percent	High
Output	REAL NEWS / FAKE NEWS	label	High
Deployment	Flask web application	application	Medium
Reproducibility	Fixed random state where applicable	setting	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Rule-based keyword filtering. It uses manually selected suspicious words or patterns. It is simple and transparent but can be bypassed by changing wording and does not learn from labelled examples.

Alternative 2: Transformer-based deep learning. It can model richer semantic context but generally requires greater computational resources and more complex deployment.

Alternative 3: Passive-Aggressive + TF-IDF. It provides a lightweight linear decision model with an update-oriented learning strategy that fits sparse text features and can be extended toward incremental learning.

Criterion	Weight	Rules	Transformer	Passive-Aggressive + TF-IDF
Performance	0.30	2.0	5.0	4.2
Compute Cost	0.20	5.0	1.5	4.8
Adaptability	0.20	1.5	4.0	4.8
Explainability	0.15	5.0	2.5	4.0
Deployment Simplicity	0.15	5.0	2.0	4.7
Weighted Score	1.00	3.35	3.30	4.49

The scores above are engineering design scores, not measured accuracy results. They are used only to document the selection rationale. The Passive-Aggressive approach is selected because it offers a strong balance between lightweight deployment, sparse-text compatibility, speed, and future incremental learning.

3.5 Selected Design & Detailed Design

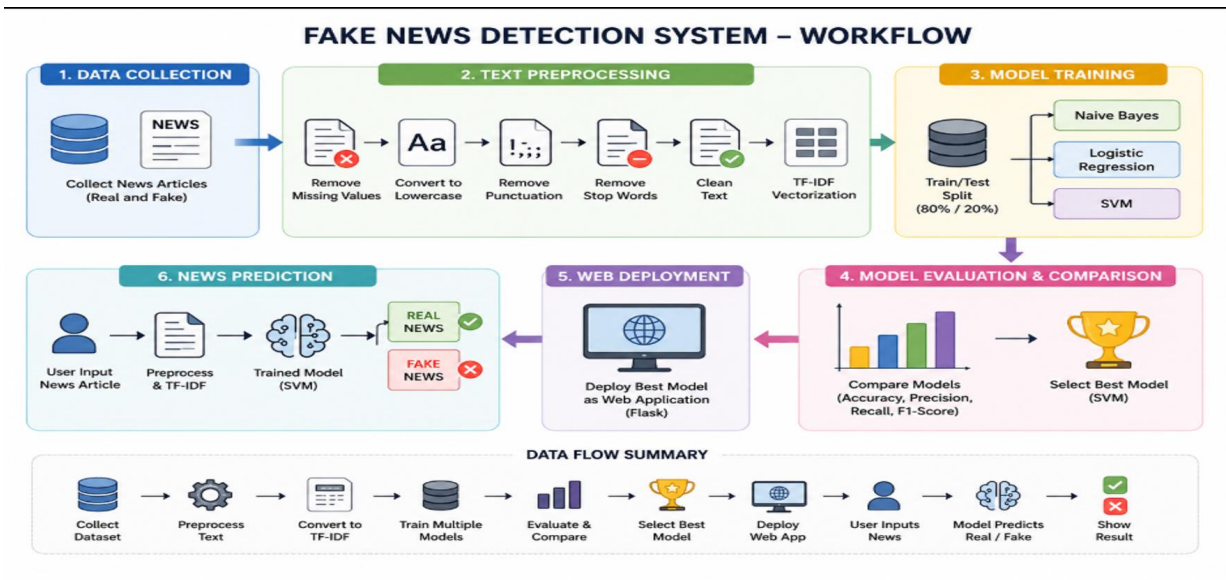


Figure 3.1: System architecture and end-to-end workflow

The system consists of six logical stages: data collection, text preprocessing, feature extraction, model training/evaluation, Flask deployment, and news prediction. The same preprocessing function and TF-IDF vectorizer must be used during training and prediction to prevent feature mismatch.

Data flows from a CSV dataset into Pandas for validation and cleaning. Cleaned text is transformed into TF-IDF vectors. The Passive-Aggressive Classifier receives the vectors and learns a decision boundary between the REAL and FAKE classes. The trained classifier and vectorizer are saved as model artifacts. During deployment, Flask loads these artifacts, preprocesses user input, transforms it using the saved vectorizer, and returns the predicted class.

The engineering design separates model training from web serving. This improves maintainability because training can be repeated without changing the web interface, while the web application can load an already-trained model for rapid prediction.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Classifier	SVM / Logistic Regression	PAC is a new algorithm and supports update-oriented learning	May be sensitive to noisy labels	Select Passive-Aggressive for algorithmic novelty and text suitability
Features	Transformer embeddings	TF-IDF is lightweight and reproducible	Less semantic understanding	Use TF-IDF for student-scale deployment
Deployment	Desktop-only script	Flask provides browser access	Requires web-server setup	Use Flask
Prediction wording	True/False	Simple	Overstates certainty	Use likely REAL/FAKE screening result

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
--------	---------------	--------------------	--------------------------

Sustainability	Avoid unnecessarily heavy models for a lightweight prototype	CPU-based TF-IDF and linear classifier are resource efficient	Supports Passive-Aggressive + TF-IDF design
Ethics	False classification can affect trust and reputation	Need for human verification	Result is framed as screening, not proof
Health & Safety	Misinformation can influence health-related decisions	High-risk claims require trusted verification	Warning message and responsible-use note included
Security	Uploaded files and user text may contain unsafe content	Input validation and controlled file handling	Validate file type, size and empty input

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect dataset labels	Medium	High	High	Inspect labels, remove invalid records, document source	Medium
Overfitting to training vocabulary	Medium	High	High	Use held-out test set and monitor metrics	Medium
False positive / false negative	Medium	High	High	Report precision, recall and F1; encourage verification	Medium
Adversarial wording / slang	Medium	Medium	Medium	Expand and refresh training data	Medium
Unsafe uploaded file	Low	High	Medium	Restrict extension/size and validate CSV structure	Low
Model artifact mismatch	Low	Medium	Low	Version model and vectorizer together	Low

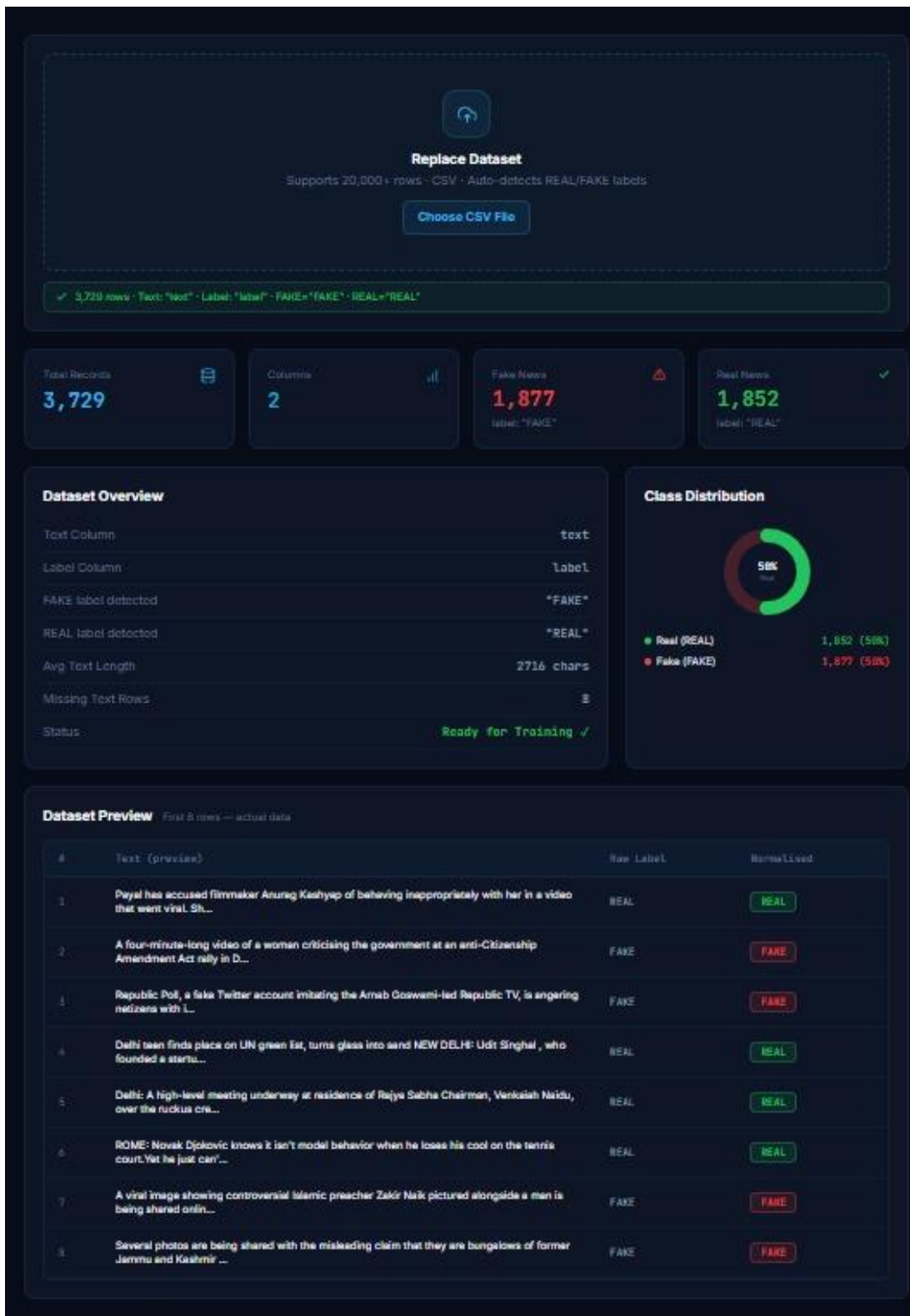


Figure 3.2: Dataset overview and class distribution interface

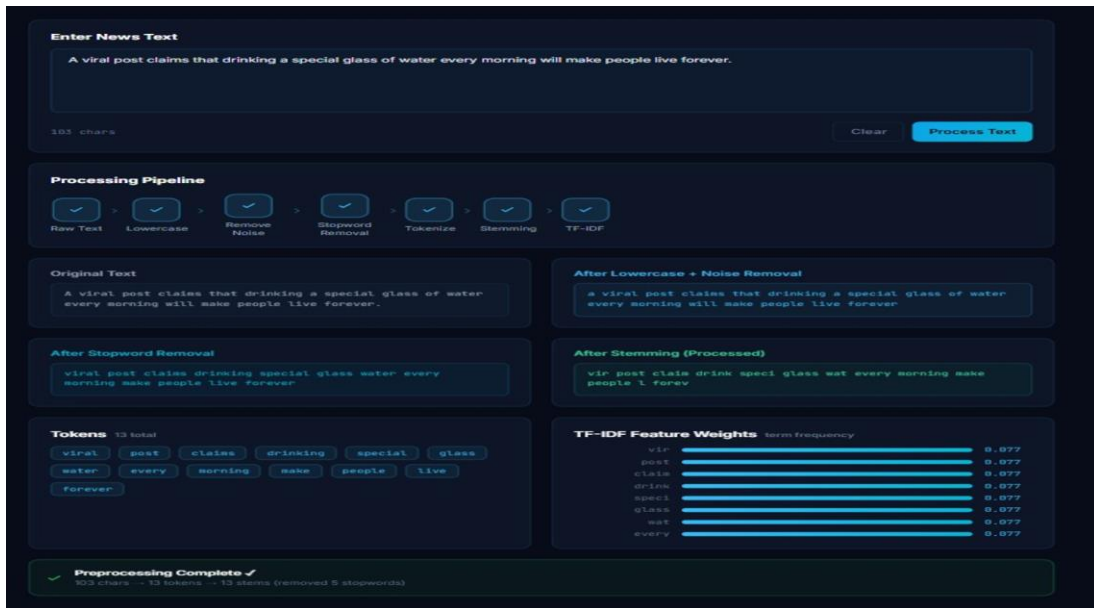


Figure 3.3: Text preprocessing and TF-IDF feature visualization interface

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows an iterative machine-learning software workflow. First, the dataset is inspected and labels are validated. Next, the text preprocessing function is implemented and reused during both training and prediction. TF-IDF is then fitted only on the training data, and the Passive-Aggressive Classifier is trained. The model is evaluated using the held-out test set and saved for deployment. Finally, the Flask application loads the saved artifacts and provides the browser-based prediction interface.

Resource	Purpose
Labelled news CSV	Training and testing data
Python 3	Programming language
Pandas / NumPy	Data loading and numerical operations
Scikit-learn	TF-IDF, Passive-Aggressive classifier and evaluation metrics
Flask	Web application and prediction API
HTML / CSS / JavaScript	User interface
VS Code	Development and debugging
Pickle / Joblib	Model artifact persistence

4.2 Software Implementation & Modern Tools Used

Tool / Software	Purpose	Stage Used
Python	Core implementation	Entire development
Pandas	CSV loading, cleaning and inspection	Data preparation
NumPy	Numerical processing	Model support
Scikit-learn	TF-IDF and Passive-Aggressive classification	Training/evaluation
Flask	Backend web service	Deployment
HTML/CSS/JavaScript	Interactive UI	Deployment
VS Code	Coding, debugging and execution	Entire development

Recommended project structure:

AI-Fake-News-Detection-System/

```
├── Dataset/
│   └── news_dataset.csv
├── static/
│   └── style.css
├── templates/
│   └── index.html
├── app.py
├── train_model.py
├── fake_news_model.pkl
├── tfidf_vectorizer.pkl
└── model_info.pkl
```

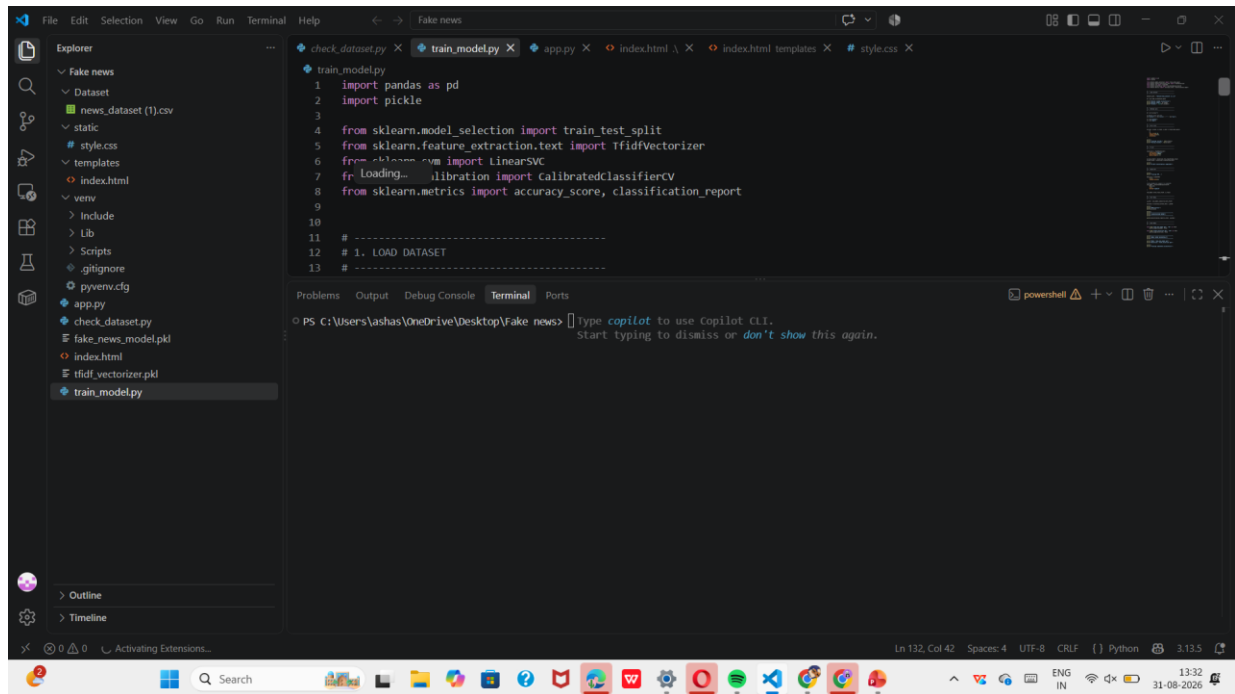


Figure 4.1: Development environment and project implementation evidence

4.3 Algorithm and Pseudocode

Selected Algorithm: Passive-Aggressive Classifier (PAC). The classifier is a linear online-learning algorithm. It is called passive when the current prediction satisfies the training constraint and aggressive when a training example violates that constraint. In practical text classification, it can learn efficiently from sparse TF-IDF vectors and can be updated with new examples when the application evolves.

9. Load the labelled news dataset.
10. Identify the text and label columns.
11. Remove missing or invalid records.
12. Normalize and clean the news text.
13. Split the dataset into training and testing subsets.
14. Fit the TF-IDF vectorizer using the training text only.
15. Transform training and testing text into TF-IDF vectors.
16. Initialize the Passive-Aggressive Classifier with a reproducible random state.
17. Train the classifier using the TF-IDF training matrix.
18. Predict labels for the test matrix.
19. Calculate accuracy, precision, recall, F1-score and confusion matrix.
20. Save the trained model and TF-IDF vectorizer.
21. Load the saved artifacts in Flask and classify new user text.

Pseudocode

BEGIN

- Load labelled news dataset
- Validate text and label columns
- Clean all news text

Split data into train and test sets
 Fit TF-IDF on training text
 Transform train and test text
 Train Passive-Aggressive Classifier
 Predict test labels
 Compute Accuracy, Precision, Recall, F1 and Confusion Matrix
 Save model and vectorizer
 Start Flask application
 Receive user news text
 Clean and TF-IDF transform input
 Predict REAL or FAKE
 Display result and warning for independent verification
 END

4.4 Test Plan & Verification

Requirement ID	Test Method	Acceptance Criterion	Status
TC01	Enter a valid real-news sample	System returns REAL or valid class output	To verify
TC02	Enter a valid fake-news sample	System returns FAKE or valid class output	To verify
TC03	Submit empty text	Validation warning is displayed	To verify
TC04	Use punctuation, URLs and special characters	Preprocessing removes supported noise safely	To verify
TC05	Use uppercase/lowercase variations	Text is normalized consistently	To verify
TC06	Load labelled CSV	Dataset records and labels are displayed	To verify
TC07	Train Passive-Aggressive model	Model and vectorizer files are created	To verify
TC08	Restart Flask app	Saved model loads without retraining	To verify
TC09	Use unseen article	Prediction is returned with responsible-use note	To verify

Specification	Required Value	Achieved Value	Status
Algorithm	Passive-Aggressive Classifier	Selected; execution required	Pending final run
Feature extraction	TF-IDF	Designed	Ready
Train/test split	80/20 recommended	Implementation requirement	Ready
Output	REAL / FAKE	Designed	Ready
Web framework	Flask	Designed	Ready
Accuracy	Actual measured value	Not supplied for new algorithm	Pending
Precision	Actual measured value	Not supplied for new algorithm	Pending
Recall	Actual measured value	Not supplied for new algorithm	Pending
F1-score	Actual measured value	Not supplied for new algorithm	Pending

4.5 Results

The earlier supplied project contains measured results for Naive Bayes, Logistic Regression, and SVM, but those algorithms are intentionally not used as the final algorithm in this revised report. The new Passive-Aggressive model has not been executed against the actual CSV within the supplied materials available for this report. Therefore, numerical performance values are intentionally left blank rather than fabricated.

Evaluation Parameter	Result for New Passive-Aggressive Model
Dataset size	Use actual project CSV count
Training set	80% of valid records

Testing set	20% of valid records
Algorithm	Passive-Aggressive Classifier
Feature extraction	TF-IDF
Accuracy	[Insert actual execution value] %
Precision	[Insert actual execution value] %
Recall	[Insert actual execution value] %
F1-Score	[Insert actual execution value] %
Confusion Matrix	[Insert actual execution matrix]

This approach follows the source project's own guidance that final reports should use actual experimental values rather than estimated values. [filecite](#)[turn10file0](#)[L22-L47](#)

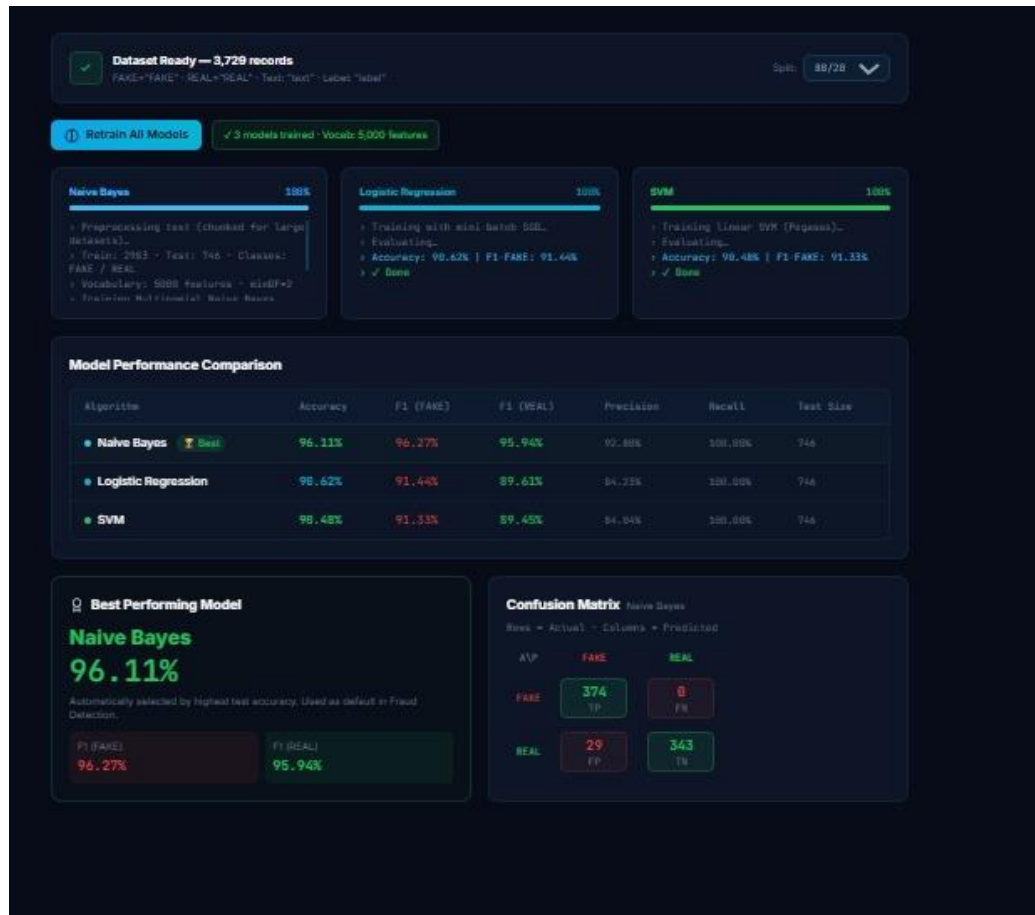


Figure 4.2: Earlier model-training dashboard used as implementation reference; final results must be regenerated for Passive-Aggressive.

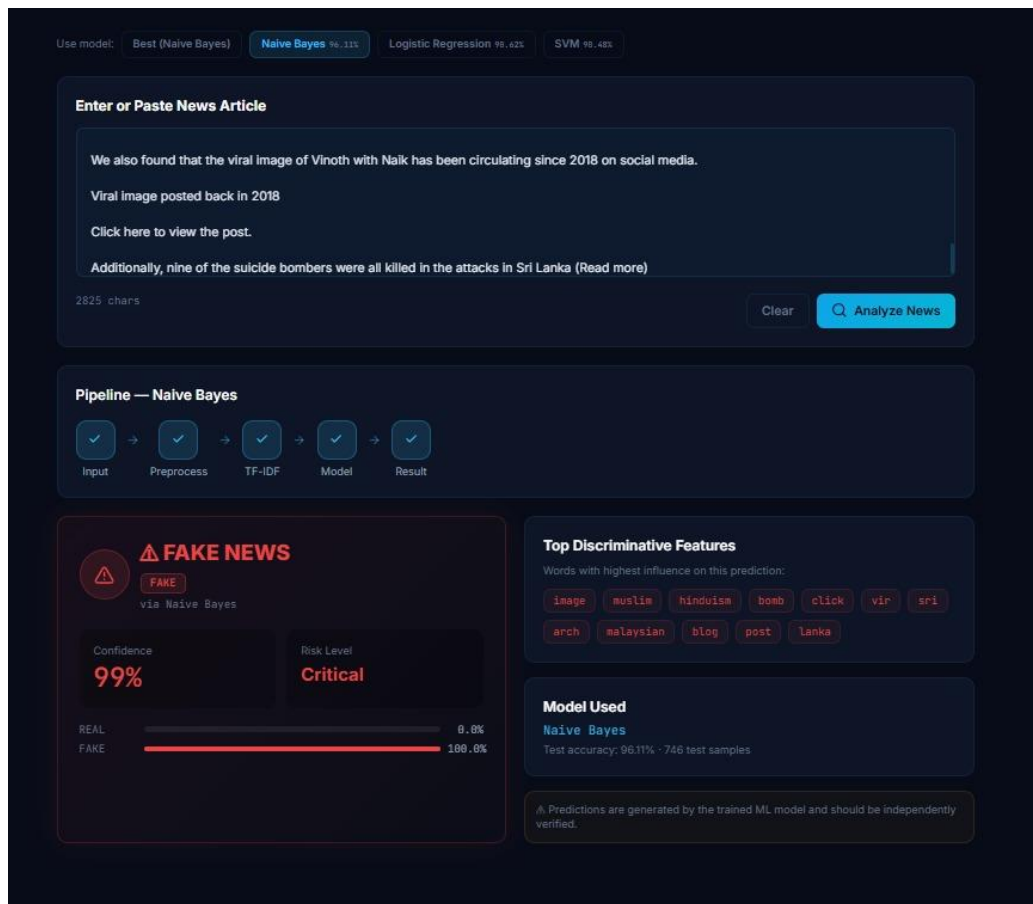


Figure 4.3: Earlier prediction interface used as UI reference; final algorithm label should be changed to Passive-Aggressive.

4.6 Data Analysis & Engineering Judgment

The principal engineering decision is to prioritize a lightweight, reproducible classifier rather than a resource-intensive deep-learning model. TF-IDF produces sparse numerical features, and the Passive-Aggressive Classifier can operate efficiently on such high-dimensional input. This is particularly useful for a student-scale web application where training and inference should remain practical on ordinary CPU hardware.

Accuracy alone is insufficient for misinformation screening. Precision measures how often predicted fake items are actually fake, while recall measures how many fake items are successfully identified. F1-score balances precision and recall. False negatives are important because a fake article incorrectly classified as real can pass through the screening stage. The final evaluation should therefore report all metrics and the confusion matrix together.

The supplied earlier report shows why metric consistency matters: it reported Logistic Regression at 98.62% accuracy, SVM at 98.48%, and Naive Bayes at 96.11%, while the dashboard labelled Naive Bayes as best. The revised project avoids carrying those old values into the new algorithm and instead requires one consistent final execution run. □ filecite □ turn10file7 □ L545-L586 □

4.7 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Prepare labelled dataset	CSV loading and validation module	Planned / verify
Preprocess text	Cleaning and normalization pipeline	Implemented design
TF-IDF feature extraction	Scikit-learn vectorizer	Implemented design
Train unique algorithm	Passive-Aggressive Classifier	Selected; final execution required
Evaluate model	Accuracy, precision, recall, F1 and confusion matrix	Pending final run
Deploy web system	Flask app and UI	Implemented design
Responsible use	Warning that prediction is not proof	Included

4.8 Strengths & Limitations

Strengths:

- Uses an algorithm different from the earlier Naive Bayes, Logistic Regression and SVM implementation.
- Lightweight and suitable for sparse TF-IDF text features.
- Can support incremental or online-style learning extensions.
- Clear separation between training and deployment.
- Browser-based interface improves accessibility.
- Provides a reproducible evaluation framework.

Limitations:

- The classifier learns from labelled examples and cannot independently verify real-world facts.
- Performance depends on dataset quality, label correctness and distribution.
- Sarcasm, emerging slang, domain shifts and adversarial wording may reduce accuracy.
- The initial scope is primarily text-based and does not verify images, videos or audio.
- Final numerical performance for the new algorithm must be generated from the actual project dataset.

4.9 Project Planning, Roles & Budget

Phase	Week / Day	Major Activity	Evidence
1	Day 1	Problem definition and dataset inspection	Dataset notes
2	Day 1	Preprocessing and TF-IDF design	Preprocessing output
3	Day 1–2	Passive-Aggressive model implementation	Training script
4	Day 2	Evaluation and error analysis	Metrics / confusion matrix
5	Day 2	Flask integration and UI testing	Screenshots
6	Day 2	Documentation and GitHub submission	Report / repository

Student	Assigned Responsibility	Actual Contribution
V. Asha Sarthiwicka	Data ingestion & preprocessing	Dataset validation, cleaning and preprocessing

Item	Estimated Cost	Actual Cost
Software / Python libraries	₹0	₹0
VS Code	₹0	₹0
Local computing	Existing resource	Existing resource

Dataset	Academic/open dataset	₹0 if freely available
Total	Minimal / existing resources	To confirm

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The AI-Powered Fake News and Misinformation Detection System is designed as an end-to-end NLP and machine-learning screening application. The project addresses the engineering problem of rapidly screening textual news by combining dataset preparation, preprocessing, TF-IDF feature extraction, Passive-Aggressive classification, evaluation, and Flask deployment.

The key distinction of this revised project is the use of the Passive-Aggressive Classifier instead of the Naive Bayes, Logistic Regression, and SVM algorithms used in the earlier version. The selected approach provides a lightweight and update-oriented learning method that is suitable for sparse text features. The system is intended to support users by identifying potentially misleading content, while independent verification remains necessary for important claims.

Final accuracy, precision, recall and F1-score must be populated after the new algorithm is trained on the actual project CSV. This preserves academic integrity and ensures that the final report reflects real experimental evidence.

5.2 Future Work

22. Use larger and more diverse datasets covering multiple domains and current misinformation patterns.
23. Introduce incremental learning so the model can be updated with newly labelled news.
24. Add multilingual NLP for Indian and international languages.
25. Integrate trusted fact-checking sources and claim verification as a second-stage module.
26. Add explainability showing influential terms or model scores without implying factual proof.
27. Extend the system to image, video and audio misinformation analysis.
28. Add source credibility and publication metadata as additional features.
29. Evaluate robustness against adversarial wording, sarcasm and topic drift.
30. Deploy the application securely with authentication, logging controls and privacy-aware storage.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical understanding of NLP preprocessing and TF-IDF feature engineering.
- Understanding of Passive-Aggressive online learning and its suitability for sparse text.
- Experience integrating a machine-learning model with Flask.
- Experience designing test cases, interpreting confusion matrices and avoiding fabricated metrics.
- Awareness of ethical limitations when AI systems classify potentially sensitive information.

Engineering & Professional Skills Developed

- Problem formulation and requirements analysis

- Algorithm selection and trade-off analysis
- Python implementation and debugging
- Team coordination and documentation
- Testing, validation and evidence-based reporting

INDIVIDUAL REFLECTION – V. Asha Sarthiwika

Register Number: 192311066

Role: Dataset and NLP preprocessing.

Most difficult engineering problem: The most difficult part was ensuring that raw news text could be converted into a consistent form without losing useful information. Different records may contain URLs, punctuation, casing differences, missing values, or unwanted characters.

Key engineering decision: I would keep a single preprocessing function shared by training and prediction so that the model never sees differently processed text at deployment time.

New knowledge acquired: I learned how dataset quality influences machine-learning results and how TF-IDF depends on consistent text preprocessing.

Redesign if repeated: I would add stronger duplicate detection, language checks, and data-quality reports before model training.

SDG connection: The project supports SDG 16 (Peace, Justice and Strong Institutions) through responsible information awareness and contributes indirectly to SDG 4 (Quality Education) by encouraging critical evaluation of digital information. The system is not a fact-checking authority; its role is to provide an automated screening signal.

CO attainment reflection: The assignment helped connect Artificial Intelligence theory with a practical engineering problem. It required problem formulation, algorithm selection, implementation, testing, analysis, documentation, and reflection. The exact course-outcome numbering should be confirmed against the official course syllabus before final submission.

REFERENCES

- [1] M. Bhardwaj and S. Rani, “A comprehensive survey on fake news detection using machine learning and deep learning approaches,” *International Journal of Information Management Data Insights*, vol. 1, no. 2, p. 100041, 2021.
 - [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
 - [3] J. Y. Khan, M. T. I. Khondaker, S. Afroz, G. Uddin, and A. Iqbal, “A benchmark study of machine learning models for online fake news detection,” *Machine Learning with Applications*, vol. 4, p. 100032, 2021.
 - [4] D. M. J. Lazer et al., “The science of fake news,” *Science*, vol. 359, no. 6380, pp. 1094–1096, 2018.
 - [5] W. McKinney, *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*, 3rd ed., O’Reilly Media, 2022.
 - [6] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
 - [7] UNESCO, Journalism, “Fake News” & Disinformation: Handbook for Journalism Education and Training, UNESCO, 2018.
 - [8] X. Zhou and R. Zafarani, “A survey of fake news: Fundamental theories, detection methods, and opportunities,” *ACM Computing Surveys*, vol. 53, no. 5, pp. 1–40, 2020.
 - [9] Scikit-learn documentation, *PassiveAggressiveClassifier / SGD-style linear classification documentation*, accessed for implementation guidance.
 - [10] Python Software Foundation, *Python documentation*, accessed for implementation and environment setup.
- Dataset reference: the project dataset is the labelled news CSV supplied by the team. The exact public dataset name/source URL should be added here by the team if the dataset was downloaded from an external repository. Do not invent a source that was not actually used.

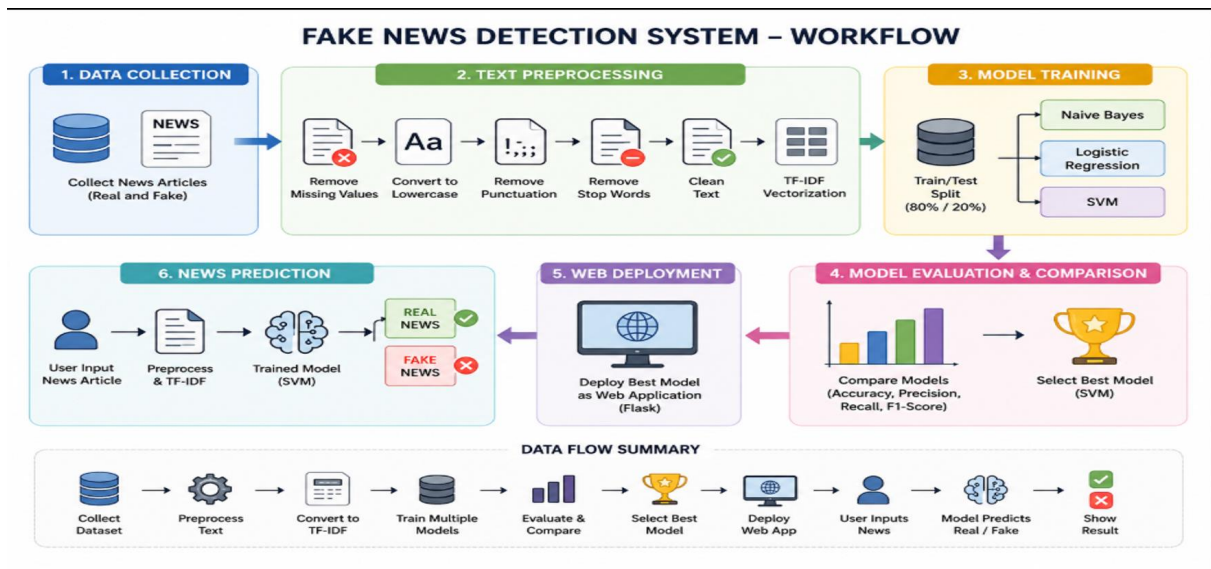
APPENDICES

Appendix A – Detailed Algorithm Notes

Passive-Aggressive learning belongs to the family of online linear classifiers. For each training instance, the algorithm checks whether the current model satisfies the classification margin. If the example is correctly classified with sufficient margin, the model remains unchanged (passive). If the constraint is violated, the model is updated (aggressive). This update-oriented behavior makes the algorithm useful for text streams and changing datasets.

For the project, the input to the classifier is a sparse TF-IDF matrix. The model can be implemented using scikit-learn's PassiveAggressiveClassifier. Hyperparameters such as C and max_iter should be selected through validation rather than by assuming one setting is always best.

Appendix B – Engineering Flowchart



Appendix Figure B.1: Project workflow reference

The flowchart should be updated in the final presentation so that the Model Training block explicitly shows Passive-Aggressive Classifier and the title matches the final project title.

Appendix C – Source Code / Repository Information

Recommended repository structure:

AI-Fake-News-and-Misinformation-Detection-System/

- Dataset/
- static/
- templates/
- app.py
- train_model.py
- fake_news_model.pkl
- tfidf_vectorizer.pkl

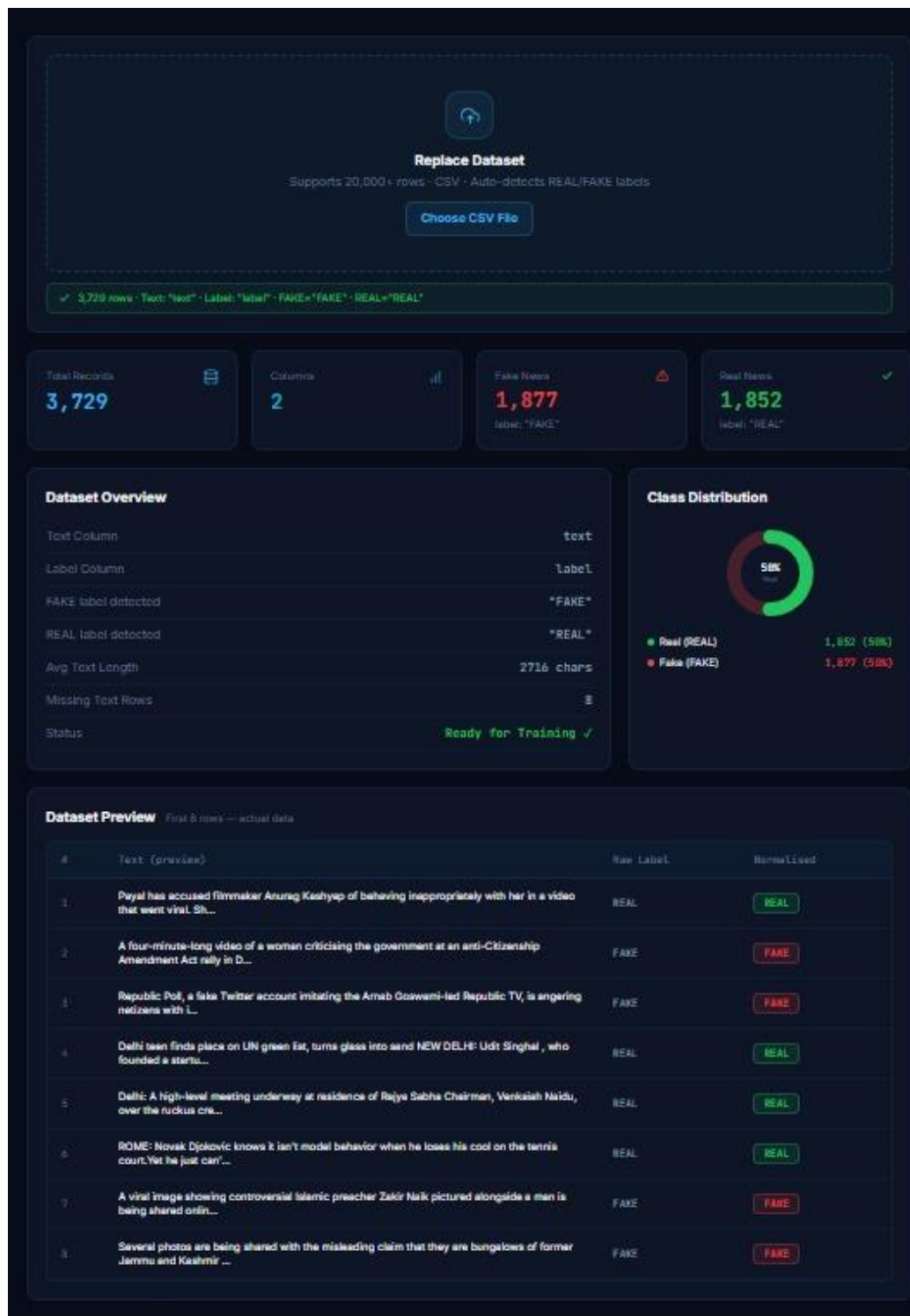
|— model_info.pkl
|— README.md

The final GitHub repository link should be inserted here and submitted through Google Classroom as required by the assignment instructions.

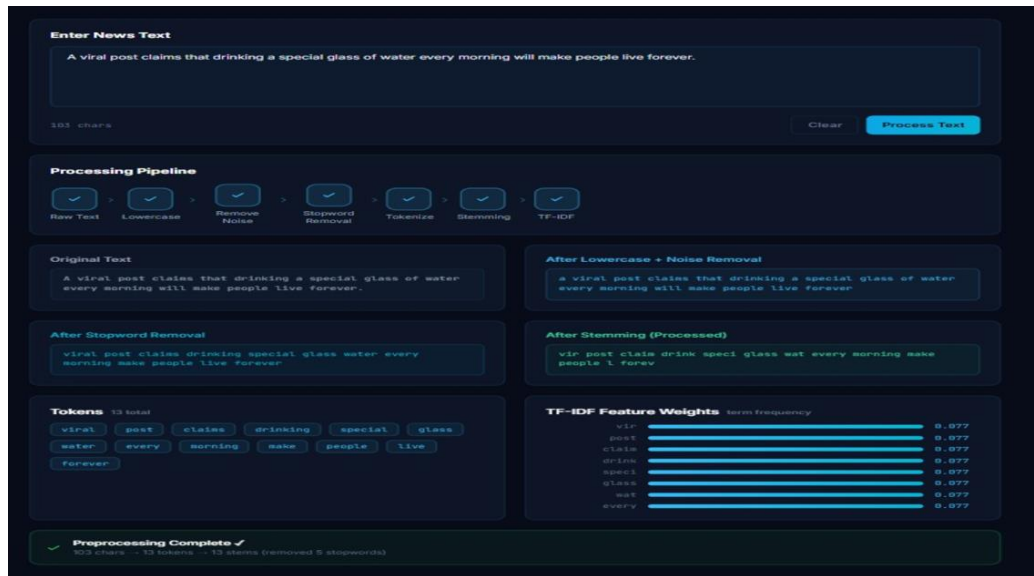
Appendix D – Experimental / Raw Data

Attach the final training console output, classification report, confusion matrix, dataset summary, and selected prediction examples after running the Passive-Aggressive model. These values should be copied directly from the execution environment.

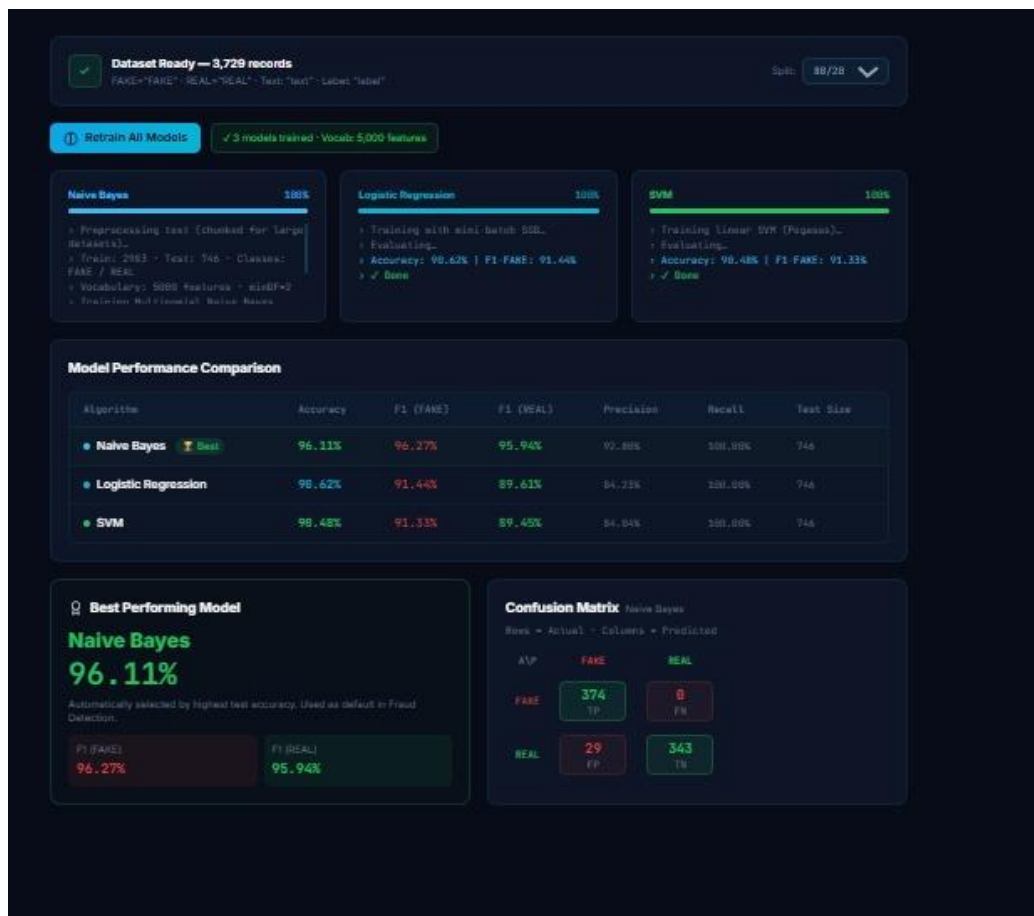
Appendix E – Execution Screenshots



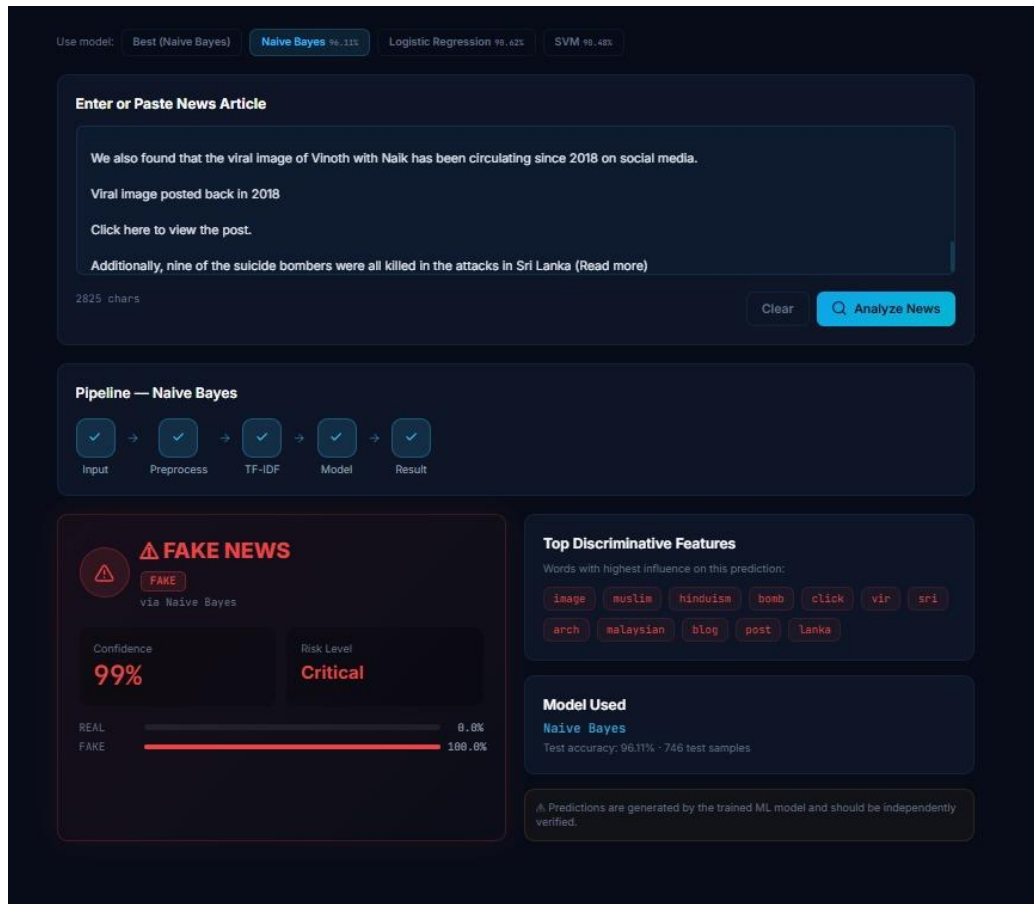
Appendix Figure E.1: Dataset dashboard evidence



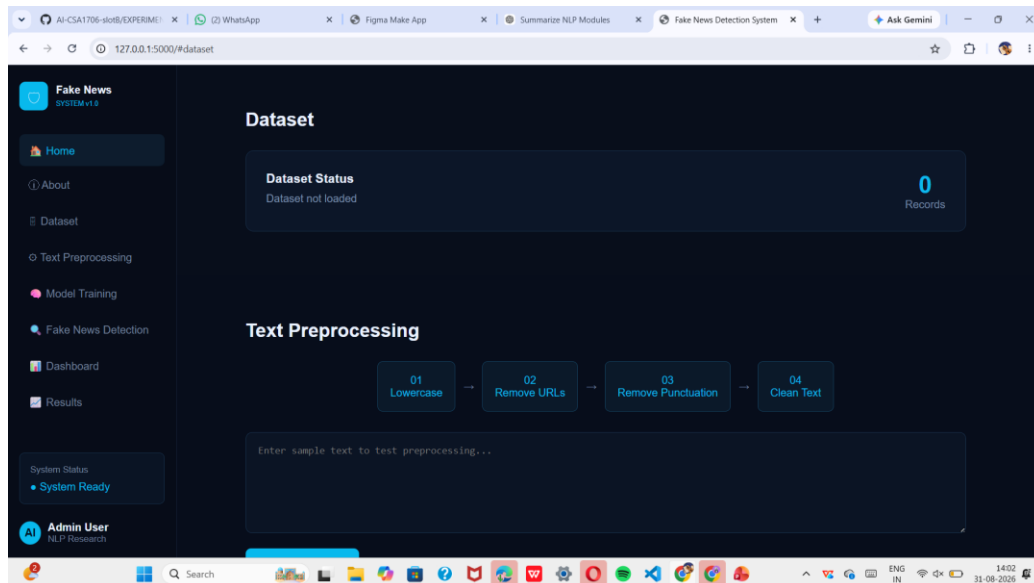
Appendix Figure E.2: Text preprocessing evidence



Appendix Figure E.3: Model-training dashboard reference



Appendix Figure E.4: Prediction interface reference



Appendix Figure E.5: Local web application development evidence

Appendix F – Ethics / Responsible AI Statement

The system is designed to assist users in identifying potentially misleading text. A classification result is a machine-learning prediction based on patterns in training data and should not be interpreted as a definitive factual judgment. The team should avoid using the system as the sole basis for decisions affecting health, safety, legal status, employment, or reputation. Important claims should be checked against reliable primary sources and professional fact-checking resources.

User-provided text should be processed only for the intended classification task. If prediction history is stored, the application should minimize personal information, apply access controls, and provide appropriate retention and deletion mechanisms.

Appendix G – CAPSTONE PROJECT OUTCOME MAPPING SHEET

The following mapping follows the structure of the supplied first-format template. The final CO numbering should be confirmed by the course faculty.

Outcome Evidence	SO / Area	Location in Report
Complex engineering problem formulation	SO1	Ch. 1 and Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3.7 and Appendix F
Teamwork and leadership	SO5	Ch. 4.9 + Contribution Statement
Experimentation, analysis and engineering judgment	SO6	Ch. 4.3–4.7
Independent acquisition of new knowledge	SO7	Ch. 5.3
Standards	SO2	Ch. 3.2
Sustainability and societal impact	SO2/SO4	Ch. 3.7
Risk assessment and mitigation	SO2/SO4	Ch. 3.7
Security considerations	Relevant SO	Ch. 3.7 and Appendix F
Integrated/system-level engineering	SO1/SO2	Ch. 3.5
Project planning and management	SO5	Ch. 4.9
Individual contribution	SO1–SO7	Contribution Statement + Ch. 5.3